

19. More pointers: least privilege and sizeof

Herman Kamper

This lecture has three parts, and they are a little disjoint. First a last example of call-by-reference, where we use the swap function from the previous lecture to build something bigger. Then the principle of least privilege, which is about the `const` keyword and what it does when you use it with pointers. And then `sizeof`, which tells you how big a box in memory actually is.

All three are worth knowing. The `const` part in particular is one of those things that looks like pedantry until you work on a program with other people, and then it turns out to be the thing that stops the program from breaking.

A last call-by-reference example

Everything here rests on the two facts from the previous lecture. When you put an `&` in front of something, you get the address of its box. When you put a `*` in front of a pointer, you go to the box it points at. There is more to pointers than that, but those two are what you use every day.

Here is the swap function again:

```
void swap(int *x, int *y)
{
    int old = *x;
    *x = *y;
    *y = old;
}
```

It takes two addresses. `*x` is the box at the first address, `*y` the box at the second. So it copies the first box's value into `old`, copies the second box into the first, and then puts the saved value into the second. It returns nothing, and yet after

```
int x = 5, y = 2;
swap(&x, &y);
printf("x = %d, y = %d\n", x, y);
```

the program prints `x = 2, y = 5`. The function reached back into main and changed both variables.

What makes this useful is that `swap` is now a piece you can build with. Anything that needs two values exchanged can call it. Let us use it for bubble sort.

Before you continue, first recap bubble sort where it was covered before. But in short, bubble sort makes repeated passes through an array. On each pass it compares neighbouring elements, and whenever the one on the right is smaller than the one on the left, it swaps them. Do that enough times and the array comes out sorted from small to large. For an array with six elements, five passes are enough.

The array we will sort is drawn below, together with the two elements being compared on one particular step.

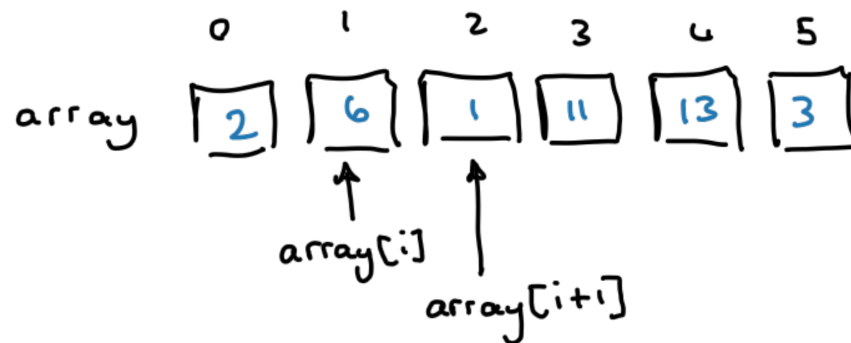


Figure 1: The array during bubble sort. On each step we compare `array[i]` with `array[i + 1]`, and swap them if the one on the right is smaller.

Here is the function, with one line left out:

```
void bubbleSort(int array[], const int size)
{
    int pass, i;

    for (pass = 0; pass < size - 1; pass++)
    {
        for (i = 0; i < size - 1; i++)
        {
            if (array[i + 1] < array[i])
            {
                // Swap values
            }
        }
    }
}
```

The outer loop counts the passes and the inner loop walks along the array. The `if` has already decided that the two elements are in the wrong order. All that is missing is the swap. Work out what that line should be before reading on.

The obvious attempt is this:

```
swap(array[i], array[i + 1]);
```

and it is wrong. Look at what swap wants. It does not take two values; it takes two addresses. And `array[i]` is not an address. It is a box, exactly like an ordinary `int x` is a box. So we have to ask for the address of that box, which means an `&` in front of each one:

```
swap(&array[i], &array[i + 1]);
```

That is the whole trick, and it is the new thing in this example: to get the address of an element of an array, you put an `&` in front of the element, not in front of the array. With that line in place the program prints

```
2 6 1 11 13 3
1 2 3 6 11 13
```

Arrays are slightly strange in this respect, because the name of an array on its own is already an address. That is why `bubbleSort(a, 6)` has no `&` in front of the `a`. Why that is so is the subject of the next lecture. For now, keep the two cases apart: array on its own is an address, while `array[i]` is a box whose address you have to ask for.

The principle of least privilege

Look at the first line of that function again:

```
void bubbleSort(int array[], const int size)
```

There is a `const` in front of `size`. You have seen the keyword before, on things like

```
const double PI = 3.14159265358979323;
```

When you have the `const` keyword, the compiler will not allow you to change that variable. For instance, if you add `size = 3;` inside the `bubbleSort` function, the program does not compile:

```
error: assignment of read-only parameter 'size'
```

Now the interesting question: why bother? The program works perfectly well without the `const`. Nothing forces you to write it, and if you simply never assign to `size`, the behaviour is identical.

The answer is that it is not there for the computer. It is there for people. Inside `bubbleSort` we change things a lot: we walk through the array, we swap elements around. But `size` should never change. Writing `const` says so, and it makes the compiler enforce it. If you later slip and write `size = i;` by accident, you are told immediately instead of spending an afternoon wondering why the loop is behaving oddly.

That is the principle of least privilege: when you write a function, give whoever calls it the least freedom they need to do the job, and give yourself the least freedom you need inside it. Be as strict as you can get away with. You are protecting yourself from other people misusing your function, and you are protecting yourself from your own future self, who will not remember in five weeks how this function was supposed to work.

It matters more the bigger the program gets. If fifty people are writing the software that runs on a satellite, being strict about what each function is allowed to touch is part of how the thing stays in space. Some languages do not offer this at all: Python has no `const`, and you can change

almost anything from almost anywhere. That is pleasant when you are writing a small script and considerably less pleasant when you are not.

const with pointers

Where this gets genuinely tricky is when you combine const with pointers, and that is worth going slowly over, because there are two separate things that could be made constant.

Remember what a pointer declaration sets up. In

```
int *myPtr = &x;
```

there are two boxes in play. There is `x`, sitting at some address, say 4004, with a value in it. And there is `myPtr`, a box of its own, holding the number 4004. Either of those boxes could be the one you want to protect: you might want to stop anyone changing the value inside `x` through this pointer, or you might want to stop the pointer from ever pointing anywhere else, or both.

C lets you write all four combinations, and the way to read them is to look at where the `const` sits relative to the `*`. The rule is in the sketch below.

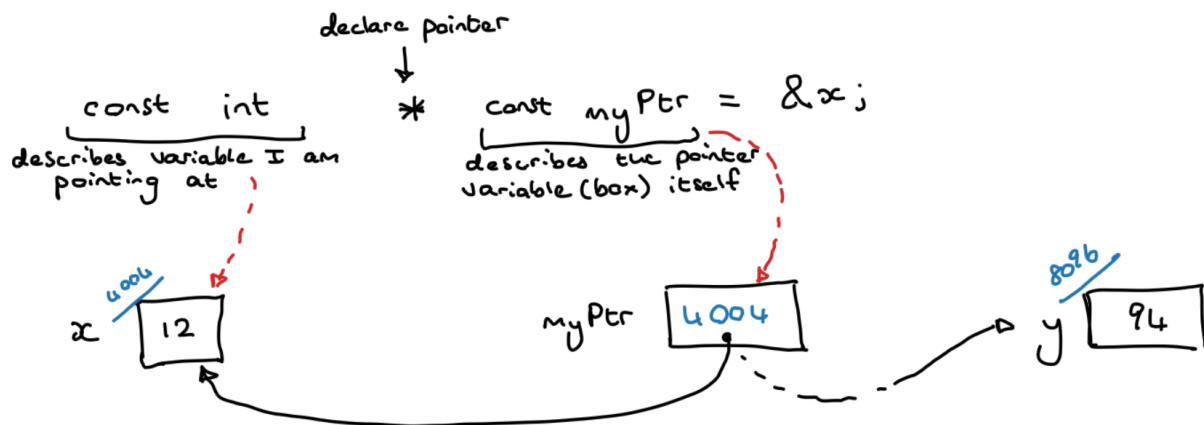


Figure 2: How to read a pointer declaration. The `*` is what declares a pointer. Everything before it describes the variable being pointed at; everything after it describes the pointer variable itself, the box holding the address.

Everything before the `*` describes the thing you are pointing at. Everything after the `*` describes the pointer box itself. Once you have that, you can work out any of the four combinations without memorising them.

Take them one at a time, assuming `int x = 12;` and `int y = 24;`.

```
int *myPtr = &x;
```

Nothing is constant. You can change `x` through the pointer with `*myPtr = 99;`, and you can point somewhere else with `myPtr = &y;`.

```
const int *myPtr = &x;
```

The `const` is before the `*`, so it describes the thing pointed at: a constant integer. You may not change the value through this pointer. But the pointer itself is an ordinary variable, so it can be made to point at something else. Trying to write through it gives:

```
error: assignment of read-only location '*myPtr'
```

```
int * const myPtr = &x;
```

Now the `const` is after the `*`, so it describes the pointer box. The address inside it is fixed: `myPtr` points at `x` and will point at `x` for as long as it exists. But `x` itself is not protected, so `*myPtr = 99;` is fine. But trying to move the pointer gives:

```
error: assignment of read-only variable 'myPtr'
```

Note that a constant pointer has to be given its value at the moment it is declared.

Let us look at the last combination:

```
const int * const myPtr = &x;
```

The pointer cannot move and the value it points at cannot be changed through it.

The table below collects the four combinations.

Declaration	Can <code>*myPtr</code> be changed?	Can <code>myPtr</code> point elsewhere?
<code>int *myPtr;</code>	yes	yes
<code>const int *myPtr;</code>	no	yes
<code>int * const myPtr;</code>	yes	no
<code>const int * const myPtr;</code>	no	no

Exam questions love this table, and if you try to memorise it you will mix up the middle two rows under pressure. Do not memorise it. Find the `*`, then read to the left for the thing pointed at and to the right for the pointer itself.

Choosing a combination

In practice these appear most often in the parameter list of a function, where you are deciding what the function is allowed to do to the caller's data. Here is a small example: a function that prints the characters of a string one at a time.

```

#include <stdio.h>

void printCharacters(const char *sPtr);

int main(void)
{
    char string[] = "print characters of a string";

    printf("The string is:\n");
    printCharacters(string);
    printf("\n");

    return 0;
}

void printCharacters(const char *sPtr)
{
    while (*sPtr != '\0')
    {
        printf("%c", *sPtr);
        sPtr++;
    }
}

```

It prints

```

The string is:
print characters of a string

```

The parameter is `const char *sPtr`: a pointer that itself can move, but pointing at characters that may not be changed in that function.

Let's unpack that a bit. The `const` is there because printing a string has no business modifying the values being printed. By writing `const`, the function advertises that it will only read, and the compiler holds it to that. Anyone calling `printCharacters` can be certain their string comes back untouched, without reading a line of the function body.

Why not go further and write `const char * const sPtr`, locking the pointer too? Because the function moves the pointer itself: `sPtr++` steps it along to the next character, which is how the loop walks through the string. (We will understand this fully in the next lecture, so don't worry about the exact details for now; just know the pointer is being changed.) Lock the pointer and the function cannot do its job:

```
error: increment of read-only parameter 'sPtr'
```

That is least privilege working as intended. You restrict everything you can, but not more than that. The data is protected because the function has no reason to write to it; the pointer is left free because stepping it along is exactly what the function is for.

There is one more thing about `sPtr++` worth flagging: incrementing a pointer moves it to the next character rather than adding 1 to an address in some naive way. Why that works, and what it has to do with arrays, is the next lecture.

How big is a box?

The third topic for this lecture ties back to something we have been saying since the first lecture. A variable is a box in memory. That is a picture, but it is also fairly literal: memory is a piece of silicon, and on it are tiny regions that hold a high or a low voltage. A one or a zero.

One of those positions is a *bit*, the smallest thing you can store, representing a single zero or one. Bits are grouped into chunks of eight, and a chunk of eight is called a *byte*. When you buy a computer with so many gigabytes of memory, that is what the word means.

So when we say a variable is a box, the natural question is how big the box is: how much of that silicon does one `int` take up? C answers this with `sizeof`, which gives you the size of something in bytes. Here it is applied to a few different variables:

```
#include <stdio.h>

int main()
{
    char c;
    int a;
    float f;
    double d;
    int x[10];
    int *aPtr = &a;
    double *dPtr = &d;

    printf("char:      %ld\n", sizeof(c));
    printf("int:       %ld\n", sizeof(a));
    printf("float:     %ld\n", sizeof(f));
    printf("double:    %ld\n", sizeof(d));
    printf("int[10]:  %ld\n", sizeof(x));
    printf("int *:    %ld\n", sizeof(aPtr));
    printf("double *: %ld\n", sizeof(dPtr));

    return 0;
}
```

Note that there is nothing in most of these boxes: `a` has never been assigned. That does not matter. The box exists from the moment it is declared, and it has a size whether or not anything useful is inside it.

On the machine I ran this on, the program prints:

```
char:      1
int:       4
float:     4
double:    8
int[10]:   40
int *:     8
double *:  8
```

Take these one at a time.

A `char` is one byte, so eight bits: eight ones and zeros on the silicon. An `int` is four bytes, which is 32 bits, since `sizeof` reports bytes and you multiply by eight to get bits. A `float` is also four bytes. A `double` takes eight bytes: it is a fatter float, storing decimal numbers more precisely, and that precision costs more ones and zeros.

The array is the nice one. `int x[10]` is ten boxes of four bytes each, and `sizeof` gives 40. That is the whole array, not one element. Make it an array of `double` and it becomes 80.

Then the pointers. Both are eight bytes. This is crucial: a pointer holds an address, and on this machine an address takes 64 bits. What it points at makes no difference at all. A pointer to a `char`, a pointer to a `double`, a pointer to an array: all eight bytes, because they are all just addresses.

The number 8 here is a property of the machine rather than of C. This is a 64-bit machine, so addresses are 64 bits (8 bytes). When I first taught this course the same program printed 4, because the machine was a 32-bit one at the time.

Looking forward

Two things to take from this lecture. The first is the principle of least privilege: use `const` to say exactly what a function is allowed to touch, and read `const` in a pointer declaration by finding the `*` and looking at what falls on each side of it. The second is `sizeof`, which tells you the size of a box in bytes.

The next lecture is about the relationship between pointers and arrays: why the name of an array is an address, why `sPtr++` steps to the next element rather than the next byte, and why `sizeof` is needed when dealing with pointers.

Exercises

1. Take the bubble sort program and change the call inside the `if` to `swap(array[i], array[i + 1])`; , without the two `&` characters. Compile it and read the error. In terms of boxes and addresses, say exactly what the compiler is objecting to.
2. For each declaration below, say whether the value pointed at can be changed, and whether the pointer can be made to point somewhere else. Then write a small program to check your answers against the compiler.

```
int a = 1, b = 2;
```

```
int *p = &a;  
const int *q = &a;  
int * const r = &a;  
const int * const s = &a;
```

3. Why does `int * const r` have to be given a value on the line where it is declared, while `int *p` does not?
4. Add `const` to the array parameter of `bubbleSort`, so that the first line reads `void bubbleSort(const int array[], const int size)`. Compile it and read what the compiler says about the call to `swap`. Explain the complaint.
5. Write a function `int countVowels(const char *sPtr)` that counts the vowels in a string. Use the same stepping pattern as `printCharacters`. Why is `const` right for this function, and why would a constant pointer be wrong?
6. Predict the output of the following, then run it:

```
char c;  
short s;  
long l;  
float f[5];  
double *dPtr;  
  
printf("%ld %ld %ld %ld %ld\n",  
       sizeof(c), sizeof(s), sizeof(l), sizeof(f), sizeof(dPtr));
```

Which of these numbers would you expect to be different on a 32-bit machine, and why?

7. An `int` takes four bytes on most machines. How many ones and zeros is that, and how many different values can be stored in that many bits? Compare your answer with the largest `int` your machine can hold.
8. Write a program that declares an array `int data[20]` and then prints, using only `sizeof`, how many elements the array has, without writing the number 20 anywhere except in the declaration. Keep your answer: this is the trick the next lecture builds on.